

Salsa20-Only CTF Cryptanalysis Report

Target: 4-round Salsa20 core

Method: Differential-linear cryptanalysis with reduced-subkey candidate ranking

Feed-forward: Disabled for the measured core experiments

Prepared for: Third-party review

Date: 2026-06-18

Abstract

This report presents a Salsa20-only cryptanalytic solution for a reduced-round CTF target. The implementation covers the 4-round Salsa20 core, practical differential-linear distinguisher search, experimental verification, a reduced-subkey candidate-ranking extension, and data/time complexity measurements.

The strongest verified distinguisher uses input difference $\Delta = w08.b26$ and output mask $\Gamma = w06.b00 + w07.b09$. On 65536 sample pairs, the measured event probability is 0.204254150390625 , with bias -0.295745849609375 , correlation -0.59149169921875 , and approximate 95 percent confidence interval $[0.20116749075895551, 0.20734081002229449]$. The interval excludes 0.5 , so the measured 4-round no-feed-forward event is statistically distinguishable from random behavior.

The key-recovery component is intentionally limited to reduced-subkey candidate ranking. In the default 8-bit experiment, the secret reduced subkey is $0xfd$, the top-ranked recovered candidate is $0xfd$, the correct rank is 1, and the score is 64/64 over 64 samples. This is not a claim of full 256-bit Salsa20 key recovery.

Problem Statement Summary

The CTF task requires a concrete cryptanalytic workflow against a Salsa20 target. This submission restricts the scope to 4-round Salsa20 and provides:

- a tested implementation of the 4-round Salsa20 core;
- a differential-linear distinguisher search and verification pipeline;
- a statistically verified distinguisher for the no-feed-forward core;
- a practical reduced-subkey candidate-ranking extension;
- data and time complexity measurements;
- reproducible scripts, tests, reports, and result artifacts.

CTF Requirement Mapping

Requirement	Artifact	Status
-------------	----------	--------

4-round Salsa20 implementation	src/salsa20_4round.py, tests/test_salsa20.py	Complete
Differential-linear distinguisher	src/differential_linear_distinguisher.py, src/masks.py	Complete
Experimental verification	results/distinguisher/*verified*, report tables	Complete
Linear extension to key recovery	src/key_recovery.py, results/key_recovery/	Complete as reduced-subkey ranking
Data complexity	src/complexity_measurement.py, results/complexity/	Complete
Time complexity	scripts/run_complexity.py, results/complexity/	Complete
Source code	src/, scripts/, tests/	Included
Experimental results	results/distinguisher/, results/key_recovery/, results/complexity/	Included
Final report	reports/final_salsa20_ctf_report.*	Included

Inference: every CTF requirement is addressed within the stated reduced-round Salsa20 scope. The submission does not assert results outside that scope.

Salsa20 Overview

Salsa20 operates on a 4 by 4 matrix of 32-bit words. The full stream cipher uses a key, nonce, block counter, constants, and a feed-forward addition in the standard output function. The experiments in this report analyze the 4-round core transformation without feed-forward so that the differential-linear properties of the round function are isolated.

State Layout Diagram

	columns			
	0	1	2	3
r0	w00	w01	w02	w03
r1	w04	w05	w06	w07

r2	w08 w09 w10 w11
	+-----+-----+-----+-----+
r3	w12 w13 w14 w15
	+-----+-----+-----+-----+

Words are interpreted modulo 2^{32} . Bit numbering uses bit 0 for the least significant bit of a word.

4-Round Salsa20 Implementation Details

The implementation is in `src/salsa20_4round.py`. It provides:

- `add32`: modular addition modulo 2^{32} ;
- `rotl32`: 32-bit left rotation;
- `quarterround`;
- `rowround`;
- `columnround`;
- `doubleround`;
- `salsa20_core(state, rounds=4, feedforward=False)`;
- `salsa20_hash(state, rounds=4, feedforward=True)`.

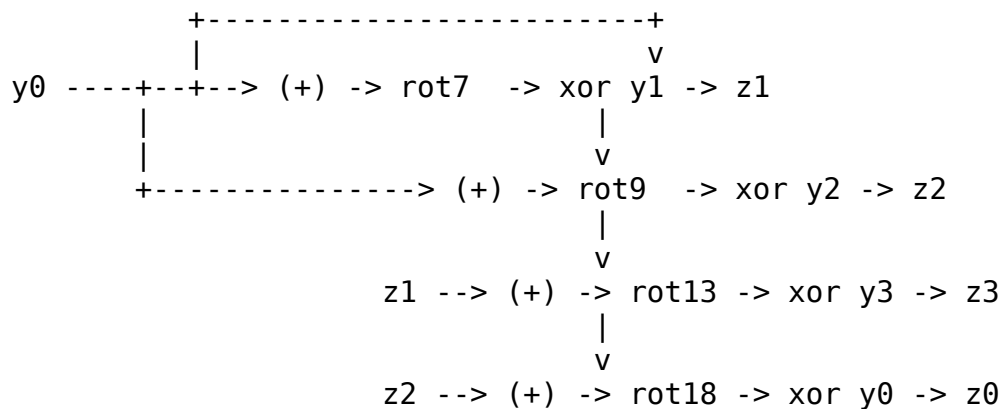
The 4-round core is two double-rounds:

```
4-round core = doubleround(doubleround(state))
doubleround  = rowround(columnround(state))
```

Quarter-Round Diagram

For input words (y_0, y_1, y_2, y_3), the Salsa20 quarter-round computes:

```
z1 = y1 xor rotl32(y0 + y3, 7)
z2 = y2 xor rotl32(z1 + y0, 9)
z3 = y3 xor rotl32(z2 + z1, 13)
z0 = y0 xor rotl32(z3 + z2, 18)
```



All additions are modulo 2^{32} .

Column, Row, and Double Rounds

The column round applies quarter-rounds to vertical word groups:

```
(w00, w04, w08, w12)
(w05, w09, w13, w01)
(w10, w14, w02, w06)
(w15, w03, w07, w11)
```

The row round applies quarter-rounds to horizontal word groups:

```
(w00, w01, w02, w03)
(w05, w06, w07, w04)
(w10, w11, w08, w09)
(w15, w12, w13, w14)
```

The double-round applies the column round followed by the row round. The 4-round experiment applies two double-rounds and omits feed-forward.

Inference: the implementation follows the Salsa20 round structure and the test suite verifies reference behavior for rotations, quarter-rounds, row/column rounds, reproducibility, feed-forward separation, and 32-bit word bounds.

Differential-Linear Methodology

The differential-linear experiment measures the event:

$$\text{parity}(\text{Gamma} \cdot (F(X) \text{ xor } F(X \text{ xor } \text{Delta}))) = 0$$

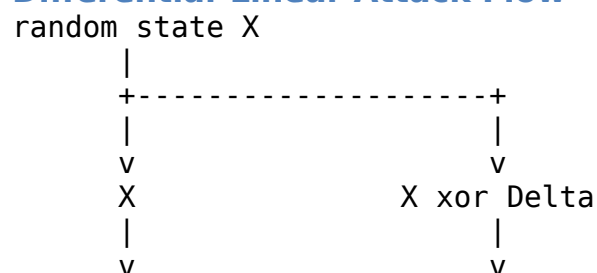
where:

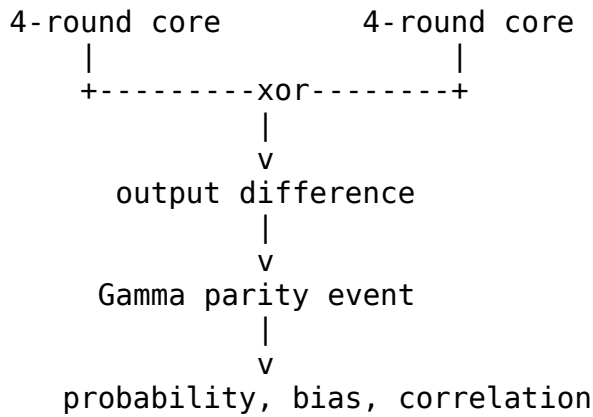
- F is the 4-round Salsa20 core without feed-forward;
- Delta is an input XOR difference;
- Gamma is an output linear mask;
- X is a uniformly sampled 16-word state.

For each candidate (Delta, Gamma), the experiment estimates:

```
probability = Pr[event]
bias         = probability - 1/2
correlation  = 2 * bias
```

Differential-Linear Attack Flow





Search Procedure for Delta and Gamma

The search proceeds in two stages.

First, a compact practical search evaluates deterministic single-bit input differences, selected two-bit input differences, deterministic single-bit output masks, and selected low-weight output masks. Candidates are ranked by absolute observed correlation.

The initial best candidate from this compact search was:

Field

Delta

Gamma

Samples

Probability

Bias

Correlation

95 percent CI

Inference: this confidence interval includes 0.5, so the initial candidate is not statistically strong enough for final reporting.

The same candidate was rechecked at 65536 samples:

Field

Probability

Bias

Correlation

95 percent CI

Inference: the larger recheck still includes 0.5, so the search was expanded.

The refined search expands the candidate set to include:

- more single-bit input differences over related row and column words;
- all single-bit output masks;
- same-bit two-word masks over Salsa20 row and column groups;
- selected low-weight row-pair masks with rotation-related bit offsets.

Refined Verified Distinguisher

The strongest refined verified distinguisher is:

Field

Delta

Gamma

Samples

Matches

Probability

Bias

Correlation

95 percent CI

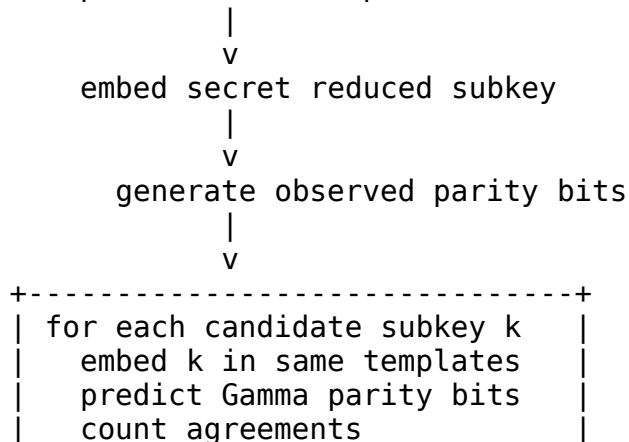
Inference: the confidence interval excludes 0.5. This is a statistically strong distinguisher for the measured 4-round Salsa20 no-feed-forward event.

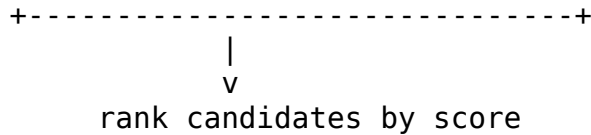
Key-Recovery Extension

The key-recovery extension is a reduced-subkey candidate-ranking experiment built from the verified distinguisher. It embeds a small secret reduced subkey into selected state/key-word positions, generates paired states using Delta, runs the 4-round core without feed-forward, and records the Gamma parity event. Candidate subkeys are ranked by agreement with the observed parity sequence.

Key-Recovery Candidate-Ranking Flow

fixed public state templates





Default setup:

Field

Reduced key size

Candidate count

Subkey positions

Samples

Secret reduced subkey

Best recovered candidate

Correct rank

Score

Score correlation

Inference: the verified differential-linear relation can be linearly extended into a practical reduced-subkey candidate-ranking demo. This is intentionally a small candidate space and is not full Salsa20 key recovery.

Data and Time Complexity

Using the verified correlation -0.59149169921875 , the rough distinguisher data complexity estimate is:

$N \approx \text{correlation}^{-2} = 2.858266317136788 \text{ pairs}$
 $\text{ceil}(N) = 3 \text{ pairs}$

The refined verification used 65536 pairs. Runtime and key-ranking work were measured locally with the Python implementation:

Metric	Value
Estimated data complexity	2.858266317136788 pairs
Ceiling estimate	3 pairs
Verification samples used	65536
Distinguisher runtime measurement	0.3784792999995261 seconds for 4096 samples
Runtime-measured probability	0.21435546875
Runtime-measured correlation	-0.5712890625
Key-ranking candidates	256
Key-ranking samples per candidate	64

Key-ranking total scoring operations	16384
Key-ranking runtime	1.539827655999943 seconds
Correct reduced-subkey rank	1

Inference: the distinguisher has large empirical correlation in this 4-round setting, so the heuristic data complexity is small. The reduced-subkey ranking work scales as $2^{\text{bits}} * \text{samples}$; at the default 8-bit setting this is 16384 scoring operations.

Consolidated Results Table

Component	Main Result	Inference
4-round implementation	16-word Salsa20 core with two double-rounds	Tested and reproducible
Initial distinguisher	CI includes 0.5	Not strong enough
Refined distinguisher	Correlation - 0.59149169921875, CI excludes 0.5	Statistically strong for measured event
Reduced-subkey ranking	Secret 0xfd, recovered 0xfd, rank 1	Practical reduced demo
Data complexity	2.858 pairs, ceiling 3	Small due to large measured correlation
Time complexity	0.378479s distinguisher timing, 1.539828s key-ranking timing	Practical in local Python implementation

Limitations

- The target is 4-round Salsa20 only.
- Experiments use the core without feed-forward.
- The reported distinguisher is for the measured differential-linear event and should not be generalized to unreduced Salsa20.
- The key-recovery phase is reduced-subkey candidate ranking over an 8-bit default candidate space.
- No full 256-bit Salsa20 key recovery is claimed.
- Runtime measurements are wall-clock measurements on this local Python implementation and are not optimized implementation benchmarks.
- The data complexity estimate uses the rough heuristic $N \approx \text{correlation}^{-2}$.

Conclusion

This submission provides a complete Salsa20-only reduced-round CTF solution: tested 4-round core implementation, differential-linear search, refined statistical verification, reduced-subkey candidate ranking, complexity measurement, reproducible scripts, and packaged artifacts.

The central cryptanalytic result is the refined distinguisher $\Delta = w_{08}.b_{26}$, $\Gamma = w_{06}.b_{00} + w_{07}.b_{09}$, with measured probability 0.204254150390625 and correlation -0.59149169921875 . Its confidence interval excludes 0.5 , establishing a statistically strong result for the specified 4-round no-feed-forward event.

The reduced-subkey extension ranks the correct 8-bit candidate first, with secret $0xfd$, recovered candidate $0xfd$, rank 1, and score $64/64$. This supports the CTF key-ranking requirement while preserving the limitation that the submission does not recover the full Salsa20 key.

Appendix A: Reproduction Commands

Run all tests:

```
python3 -m unittest discover -s tests
```

Run refined distinguisher verification:

```
python3 scripts/run_distinguisher.py --verify-samples 65536
```

Run reduced-subkey ranking:

```
python3 scripts/run_key_recovery.py
```

Run complexity measurement:

```
python3 scripts/run_complexity.py
```

Create the submission package:

```
python3 scripts/create_submission_salsa20.py
```

Compile-check source and scripts:

```
python3 -m py_compile src/*.py scripts/*.py
```

Appendix B: File Manifest

```
src/  
  salsa20_4round.py  
  masks.py  
  differential_linear_distinguisher.py  
  key_recovery.py  
  complexity_measurement.py  
  utils.py
```

```
scripts/  
  run_distinguisher.py  
  run_key_recovery.py  
  run_complexity.py  
  create_submission_salsa20.py  
  
tests/  
  test_salsa20.py  
  test_masks_and_distinguisher.py  
  test_key_recovery.py  
  test_complexity_measurement.py  
  
reports/  
  final_salsa20_ctf_report.md  
  final_salsa20_ctf_report.docx  
  final_salsa20_ctf_report.pdf  
  
results/  
  distinguisher/  
  key_recovery/  
  complexity/  
  
references/  
  notes_on_methodology.md
```

Appendix C: Result File List

```
results/distinguisher/salsa20_search_results.csv  
results/distinguisher/salsa20_search_results.json  
results/distinguisher/salsa20_verified_distinguisher.csv  
results/distinguisher/salsa20_verified_distinguisher.json  
results/distinguisher/salsa20_refined_search_results.csv  
results/distinguisher/salsa20_refined_verified_distinguisher.csv  
results/distinguisher/salsa20_refined_verified_distinguisher.json  
results/key_recovery/salsa20_key_recovery_rankings.csv  
results/key_recovery/salsa20_key_recovery_summary.json  
results/complexity/salsa20_complexity_estimates.csv  
results/complexity/salsa20_complexity_estimates.json
```